

1- وصف النظام .

نظام Book Tracker هو نظام متكامل لجمع بيانات الكتب من موقع المكتبة على الإنترنت باستخدام تقنيات web scraping، ثم معالجتها وتخزينها وتنظيمها وتخزينها داخل قاعدة بيانات PostgreSQL، وبعد ذلك توفيرها عبر واجهات Rest API باستخدام إطار العمل FastAPI.

تم تجهيز النظام ليتم كونه مستقلة داخل نظام Ubuntu ويتم التحكم به عبر systemd، مع إمكانية نشره وتشغيله على أي جهاز Ubuntu من خلال حزمة (Debian Package).

2- الهدف من المشروع .

يهدف المشروع إلى تحويل سكربت scraping بسيط إلى نظام Deployable Production-like متكامل يعمل:

- جمع البيانات اليومية من الإنترنت.
- تخزينها في قاعدة بيانات آمنة.
- توفيرها عبر API شبكة.
- تشغيلها كخدمة مستقلة في الخلفية.
- نشرها عبر حزمة تثبيت جاهزة.

٣- الازدواج والتقنيات المستخدمة

python

اللغة الأساسية المستخدمة لبناء النظام بالكامل

سبب الاختيار:

- مناسبة جداً لأعمال Web Scraping.
- تدعم fastAPI بشكل ممتاز.
- توفر مكتبات قوية للتعامل مع الشبكات وفهم البيانات

BeautifulSoup/Requests

مستخدمة في طبقة ال Scraping

وظيفة:

- إرسال HTTP Requests للموقع المستهدف.
- تحليل HTTP واستخدام البيانات المطلوبة.

PostgreSQL

قاعدة البيانات الرئيسية للمشروع.

سبب الاختيار:

- قاعدة بيانات Production Grade.
- تدعم العلاقات وال constraints بشكل قوي.
- مناسبة أكثر من SQLite للشاريع الشبكية.
- مستخدمة بكثرة في أنظمة الحفظة.

SQLAlchemy ORM layer بين Python و PostgreSQL

وظائفها:

- تسهيل التعامل مع قاعدة البيانات بطريقة Object oriented.
- تقليل كتابة SQL الخام.
- تنفيذ ال Models بشكل آمن.

FastAPI

إطار العمل المستخدم لبناء Rest API
سريع الاختيار

- سريع جداً (High Performance).
- يدعم Async.
- يولد Swagger Documentation تلقائياً.
- مناسب جداً لبناء API حديثة.

Uvicorn

Asynchronous Server لتشغيل تطبيق FastAPI

وظائفها:

- تشغيل التطبيق كـ Web server.
- استقبال HTTP Requests وتوجيهها للتطبيق.

Systemd
مستخدم لتحويل التطبيق إلى Linux Service

وظائفه

- تشغيل التطبيق في الخلفية.
- إعادة التشغيل التلقائي عند إقلاع النظام.
- التشغيل التلقائي عند إقلاع النظام.

UFW Firewall

مستخدم لفتح مقل التطبيق.

السماح بالوصول للتطبيق عبر الشبكة على Port 8000.

Debian Packaging

مستخدم لتحويل المشروع إلى حزمة دبيان.

وظائفه

- تسهيل نشر المشروع على أي Ubuntu Server.
- تنفيذ الإعدادات تلقائياً بعد التثبيت.

Date

No.

معمارية النظام

External Website



Web scraper Layer



Data cleaning Layer



Postgresql Database



FastAPI Application



REST API Endpoints



Client Application

```
abdullah@scraper-server:~$ tree -L 4 -I "venv|__pycache__"
.
├── book-tracker
│   ├── api
│   │   ├── __init__.py
│   │   └── main.py
│   ├── db
│   │   ├── database.py
│   │   ├── __init__.py
│   │   └── models.py
│   ├── package-build
│   │   ├── DEBIAN
│   │   │   ├── control
│   │   │   └── postinst
│   │   ├── etc
│   │   │   └── systemd
│   │   └── opt
│   │       └── book-tracker
│   ├── package-build.deb
│   ├── requirements.txt
│   ├── scraper
│   │   ├── __init__.py
│   │   └── scraper.py
│   ├── systemd
│   │   └── book-tracker.service
│   └── book-tracker.tar.gz
├── 12 directories, 13 files
└── abdullah@scraper-server:~$
```

لماذا قمنا باختيار هذا التقسيم للمشروع

تم تقسيم مشروع **Book Tracker** إلى عدة أجزاء (Modules) بحيث تكون كل طبقة مسؤولة عن وظيفة محددة داخل النظام، وذلك بهدف بناء نظام منظم وقابل للتطوير والصيانة.

هذا التقسيم لم يكن عشوائياً، بل تم اعتماده بناءً على مبادئ هندسة البرمجيات الحديثة.

أولاً: مبدأ فصل المسؤوليات (Separation of Concerns)

تم تقسيم المشروع إلى ثلاث طبقات رئيسية:

- طبقة جمع البيانات (Scraper)
- طبقة قاعدة البيانات (Database)
- طبقة الواجهة البرمجية (API)

الهدف من هذا الفصل هو أن يكون لكل جزء مسؤولية واضحة:

- ال Scraper مسؤول فقط عن جلب البيانات
- قاعدة البيانات مسؤولة عن التخزين
- ال API مسؤول عن عرض البيانات للمستخدم

هذا يمنع تداخل المهام ويجعل الكود أكثر وضوحًا.

ثانيًا: سهولة الصيانة (Maintainability)

في هذا التصميم، إذا حصلت مشكلة:

- في جلب البيانات → نعدل في scraper فقط
- في قاعدة البيانات → نعدل في db فقط
- في ال API → نعدل في api فقط

بدون الحاجة لتعديل النظام بالكامل.

وهذا يقلل الأخطاء ويسهل تطوير المشروع لاحقًا.

ثالثًا: قابلية التوسع (Scalability)

هذا التقسيم يسمح لنا مستقبلاً بـ:

- تغيير مصدر البيانات بدون التأثير على API
- استبدال PostgreSQL بقاعدة بيانات أخرى بسهولة
- إضافة Endpoints جديدة بدون التأثير على باقي النظام

بمعنى أن النظام قابل للنمو بدون إعادة بنائه من الصفر.

رابعًا: إعادة الاستخدام (Reusability)

كل جزء من المشروع يمكن استخدامه بشكل مستقل:

- يمكن استخدام ال Scraper في مشروع آخر

- يمكن استخدام API مع مصدر بيانات مختلف
- يمكن ربط نفس قاعدة البيانات مع أكثر من تطبيق

خامسًا: التوافق مع الأنظمة الحقيقية (Real-World Architecture)

هذا النوع من التقسيم يُستخدم فعليًا في الأنظمة الحقيقية (Production Systems) ، حيث يتم فصل:

- Data Layer
- Business Logic
- API Layer

وبالتالي المشروع يحاكي بيئة عمل حقيقية وليس مجرد سكربت بسيط.

سادسًا: تسهيل التحويل إلى Service

عند تحويل المشروع إلى **Linux Service** باستخدام **systemd** ، كان من السهل تحديد نقطة تشغيل واحدة وهي:

```
uvicorn api.main:app
```

وذلك لأن:

- الكود منظم
- نقطة الدخول واضحة
- لا يوجد تداخل بين المكونات

سابعًا: دعم عملية التغليف (Packaging)

عند بناء حزمة **.deb**:

- تم نقل المشروع إلى `/opt/book-tracker` بشكل منظم
- تم تضمين كل جزء في مكانه الصحيح

- تم تشغيله مباشرة بدون مشاكل

وهذا لم يكن ممكناً لو كان المشروع عشوائياً.

الخلاصة

تم اختيار هذا التقسيم لأنه يحقق:

- تنظيم واضح للكود
- سهولة الصيانة
- قابلية التوسع
- محاكاة الأنظمة الحقيقية
- دعم تشغيل المشروع كخدمة
- تسهيل نشره كحزمة تثبيت

وبالتالي تم تحويل المشروع من سكربت بسيط إلى نظام Backend متكامل قابل للاستخدام الفعلي.

```
GNU nano 7.2                                scraper.py *
import requests_
from bs4 import BeautifulSoup
import re
from db.database import SessionLocal
from db.models import Book
session = SessionLocal()
BASE_URL = "https://books.toscrape.com/catalogue/page-{}.html"

def clean_price(price_text):
    cleaned = re.sub(r"[^0-9.]", "", price_text)
    return float(cleaned)

print("Starting scraping...")

for page in range(1, 4):
    url = BASE_URL.format(page)
    response = requests.get(url)

    soup = BeautifulSoup(response.text, "html.parser")
    books = soup.find_all("article", class_="product_pod")

    for b in books:
        title = b.h3.a["title"]

        price_text = b.find("p", class_="price_color").text
        price = clean_price(price_text)

        rating_class = b.p["class"][1]
        rating = {
            "One": 1,
            "Two": 2,
            "Three": 3,
            "Four": 4,
            "Five": 5
        }.get(rating_class, 0)

        book = Book(
            title=title,
            price=price,
            rating=rating
        )

        session.add(book)
    session.commit()
    session.close()
print("Done + Data inserted into PostgreSQL")

G Help      O Write Out  W Where Is    C Cut         E Execute    L Location   M-U Undo      M-A Set Mark  M-M To Bracket  M-P Previous
X Exit      R Read File   R Replace    U Paste       J Justify    G Go To Line M-E Redo      M-G Copy      W Activate Windows
                                                     Go to Settings to activate Windows
```

```
ubuntu-server-project [Running] - Oracle VirtualBox
File  Machine  View  Input  Devices  Help

GNU nano 7.2                                /etc/systemd/system/book-tracker.service
[Unit]
Description=Book Tracker FastAPI Service
After=network.target

[Service]
User=abdullah
WorkingDirectory=/home/abdullah/book-tracker
ExecStart=/home/abdullah/book-tracker/venv/bin/python -m uvicorn api.main:app --host 0.0.0.0 --port 8000
Restart=always

[Install]
WantedBy=multi-user.target
```

```
GNU nano 7.2 db/models.py
from sqlalchemy import Column, Integer, String, Float
from .database import Base

class Book(Base):
    __tablename__ = "books"
    id = Column(Integer, primary_key=True, index=True)
    title = Column(String)
    price = Column(Float)
    rating = Column(Integer)
```

Mouse integration ...
Don't show again

Auto capture keyboard ...
Don't show again

Read 10 lines

Help Write Out Where Is Cut Execute Location Undo Set Mark To Bracket Previous
Exit Read File Replace Paste Justify Go To Line Redo Copy

Activate Windows
Go to Settings to activate Windows

Right Control

```
GNU nano 7.2 database.py
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, declarative_base

DATABASE_URL = "postgresql+psycopg2:///bookuser:123456789@localhost:5432/bookdb"

engine = create_engine(DATABASE_URL)

SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False)
Base = declarative_base()
```

Mouse integration ...
Don't show again

Auto capture keyboard ...
Don't show again

Response body

```
[
  {
    "title": "A Light in the Attic",
    "rating": 3,
    "id": 3,
    "price": 51.77
  },
  {
    "title": "Tipping the Velvet",
    "rating": 3,
    "id": 2,
    "price": 53.74
  },
  {
    "title": "Soumission",
    "rating": 1,
    "id": 3,
    "price": 50.1
  },
  {
    "title": "Sharp Objects",
    "rating": 4,
    "id": 4,
    "price": 47.82
  },
  {
    "title": "Sapiens: A Brief History of Humankind",

```



Download

Response headers

```
content-length: 5146
content-type: application/json
date: Thu, 16 Apr 2026 15:53:31 GMT
server: uvicorn
```

```
GNU nano 7.2                                api/main.py
)
def get_db():
    return SessionLocal()

@app.get("/health")
def health_check():
    return {"status": "healthy"}

@app.get("/books")
def get_books():
    db = get_db()
    books = db.query(Book).all()
    db.close()
    return books

@app.get("/books/{book_id}")
def get_book(book_id: int):
    db = get_db()
    book = db.query(Book).filter(Book.id == book_id).first()
    db.close()

    if not book:
        raise HTTPException(status_code=404, detail="Book not found")

    return book

@app.get("/stats")
def get_stats():
    db = get_db()

    total_books = db.query(Book).count()
    avg_price = db.query(func.avg(Book.price)).scalar()

    db.close()

    return {
        "total_books": total_books,
        "average_price": round(avg_price, 2) if avg_price else 0
    }

[ Soft wrapping of overlong lines disabled ]
Help      Write Out  Where Is  Cut       Execute   Location  M-U Undo  M-A Set Mark
Exit      Read File  Replace   Paste     Justify   Go To Line M-B Copy  M-E Redo

Activate Windows
Where's the key to activate Windows?
Right Control
```

```
GNU nano 7.2 requirements.txt
annotated-doc==0.4
annotated-types==0.7.0
anyio==4.13.0
beautifulsoup4==4.14.3
certifi==2026.2.25
charset-normalizer==3.4.7
click==8.3.2
fastapi==0.135.3
greenlet==3.4.0
h11==0.16.0
idna==3.11
lxml==6.0.4
psycopg2-binary==2.9.11
pydantic==2.13.0
pydantic_core==2.46.0
requests==2.33.1
soupsieve==2.8.3
SQLAlchemy==2.0.49
starlette==1.0.0
typing-inspection==0.4.2
typing_extensions==4.15.0
urllib3==2.6.3
uvicorn==0.44.0
```

```
GNU nano 7.2 api/main.py
from fastapi import FastAPI, HTTPException
from db.database import SessionLocal
from db.models import Book
from sqlalchemy import func

app = FastAPI(
    title="Book Tracker API",
    description="API for scraped books data",
    version="1.0.0"
)

def get_db():
    return SessionLocal()

@app.get("/health")
def health_check():
    return {"status": "healthy"}

@app.get("/books")
def get_books():
    db = get_db()
    books = db.query(Book).all()
    db.close()
    return books

@app.get("/books/{book_id}")
def get_book(book_id: int):
    db = get_db()
    book = db.query(Book).filter(Book.id == book_id).first()
    db.close()

    if not book:
        raise HTTPException(status_code=404, detail="Book not found")

    return book

@app.get("/stats")
def get_stats():
    db = get_db()

    total_books = db.query(Book).count()
```

```
GNU nano 7.2 package-build/DEBIAN/postinst
# !/bin/bash

apt update
apt install -y python3 python3-pip

pip3 install fastapi uvicorn sqlalchemy psycpg2-binary requests beautifulsoup4

systemctl daemon-reload
systemctl enable book-tracker
systemctl restart book-tracker
```

```
GNU nano 7.2 package-build/etc/systemd/system/book-tracker.service
[Unit]
Description=Book Tracker FastAPI Service
After=network.target

[Service]
User=root
WorkingDirectory=/opt/book-tracker
ExecStart=/usr/bin/python3 -m uvicorn api.main:app --host 0.0.0.0 --port 8000
Restart=always

[Install]
WantedBy=multi-user.target
```

```
GNU nano 7.2 book-tracker.service
[Unit]
Description=Book Tracker FastAPI Service
After=network.target

[Service]
User=abdullah
WorkingDirectory=/home/abdullah/book-tracker
ExecStart=/home/abdullah/book-tracker/venv/bin/python -m uvicorn api.main:app --host 0.0.0.0 --port 8000
Restart=always

[Install]
WantedBy=multi-user.target
```

```
GNU nano 7.2 package-build/DEBIAN/control
Package: book-tracker
Version: 1.0
Section: utils
Priority: optional
Architecture: all
Maintainer: Abdullah
Description: Book Tracker Web Scraping API Service
```


Name

Description

book_id required

integer

(path)

11

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/books/11' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/books/11
```

Server response

Code

Details

200

Response body

```
{
  "title": "Starving Hearts (Triangular Trade Trilogy, #1)",
  "rating": 2,
  "id": 11,
  "price": 13.99
}
```

Response headers

GET /books Get Books

Parameters

No parameters

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/books' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/books
```

Server response

Code

Details

200

Response body

```
{
  "title": "A Light in the Attic",
  "rating": 3,
  "id": 1,
  "price": 51.77
},
{
  "title": "Tipping the Velvet",
  "rating": 1,

```